

L03: Multilayer Perceptron (MLP)

FICT, UTAR

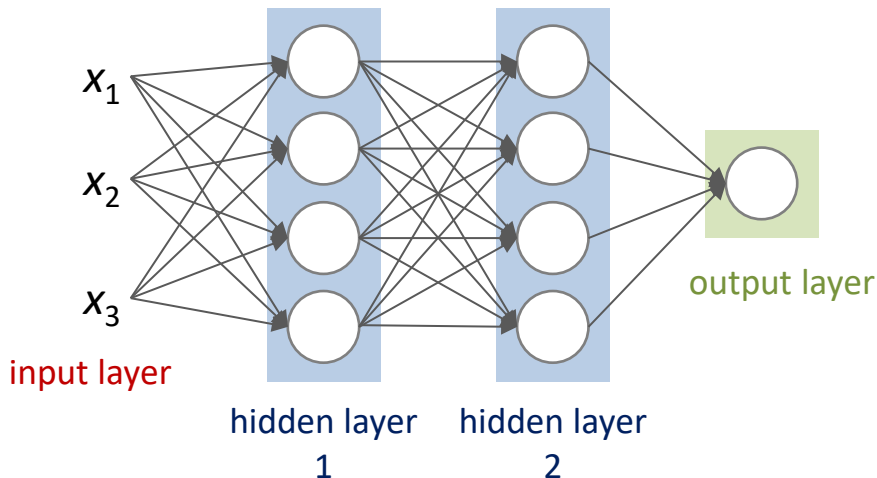
Table of Contents

1. The Multilayer Perceptron (MLP) Architecture
2. Designing the output layer
3. Shallow Neural Network
4. Deep Neural Network

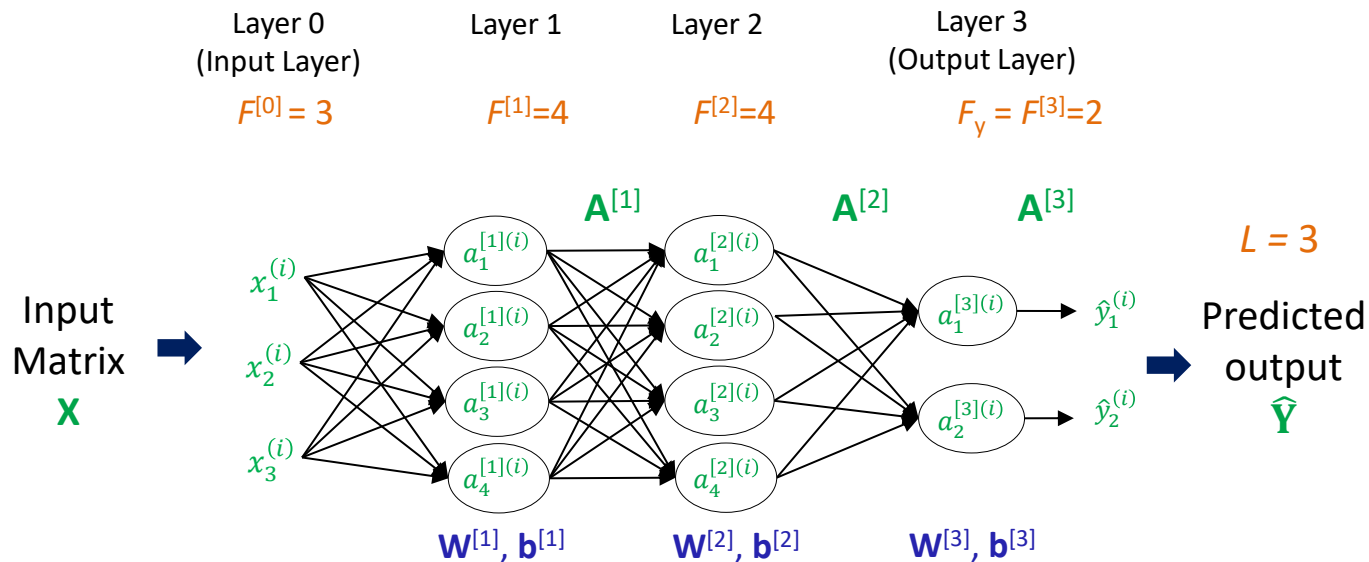
The Multilayer Perceptron (MLP) Architecture

Multilayer Perceptron (MLP)

- **Multilayer Perceptron** (MLP) is a feedforward neural network constructed by stacking **fully connected** (FC) or **Linear** layers.
- In a fully connected layer, all neurons in a current layer are fully connected to all neurons in the previous one.
- The **input layer** feeds the network with the input features.
- The **hidden layers** (1st to 2nd last layers) extract features hierarchically.
- The **output layer** (last layer) computes the output score.



Notations



B : batch size

L : #layers

$F^{[l]}$: #nodes in layer l

F_x : input dimension

F_y : output dimension

$\mathbf{X}$: input matrix (features) for all samples

$x_j^{(i)}$: input feature j for sample i

$\hat{\mathbf{Y}}$: predicted output

$\hat{y}_j^{(i)}$: predicted score for class j for sample i

$\mathbf{A}^{[l]}$: activation output of layer l for all samples

$a_j^{[l](i)}$: activations of the neuron j in layer l for sample i

$\mathbf{W}^{[l]}$: weights for layer l

$\mathbf{b}^{[l]}$: bias for layer l

$\mathbf{W}$: $\{\mathbf{W}^{[l]}\}_{l=1}^L$

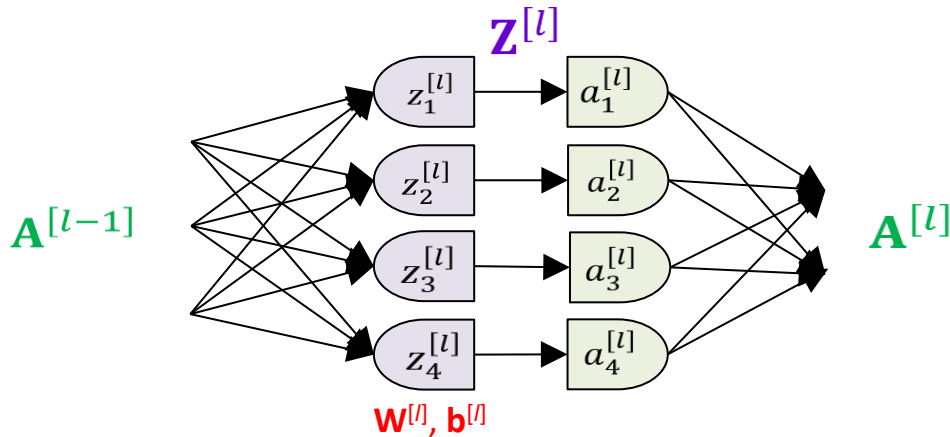
$\mathbf{b}$: $\{\mathbf{b}^{[l]}\}_{l=1}^L$

Fully Connected Layer

- A linear layer consists of **summation** followed by **activation** phase.

Summation phase

Activation phase



$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

(uses a linear layer to transform the signal)

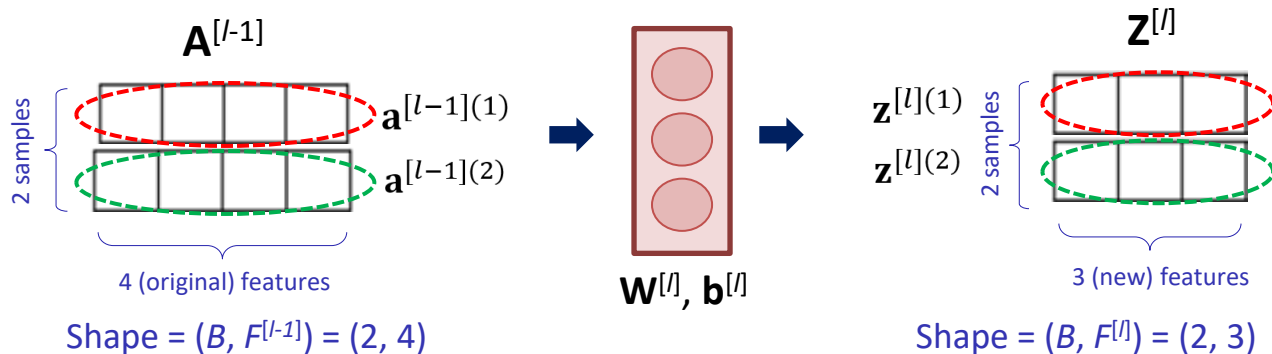
$$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$$

(Example activation functions: sigmoid, relu, gelu, elu, etc.)

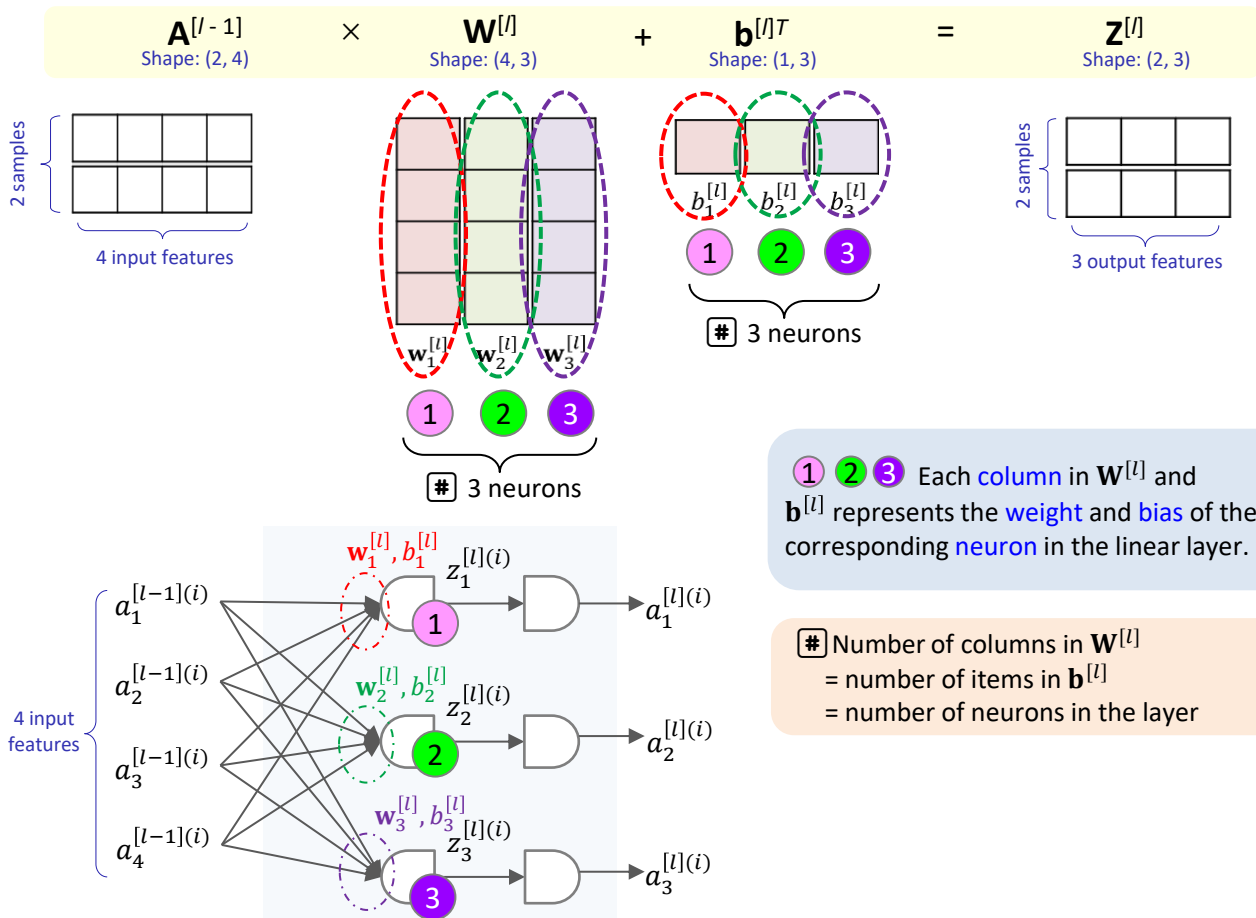
Summation Phase

- The summation phase is implemented with a **linear** layer (also called a **fully connected** layer or **dense** layer).
- The linear layer receives an **1D input vector** of dimension $F^{[i-1]}$ and **linearly transform** it to an output vector of dimension $F^{[i]}$.
- The batch samples and neurons can be processed **in parallel** where multiple samples are **batched** into a 2-D matrix of shape (B, F) .

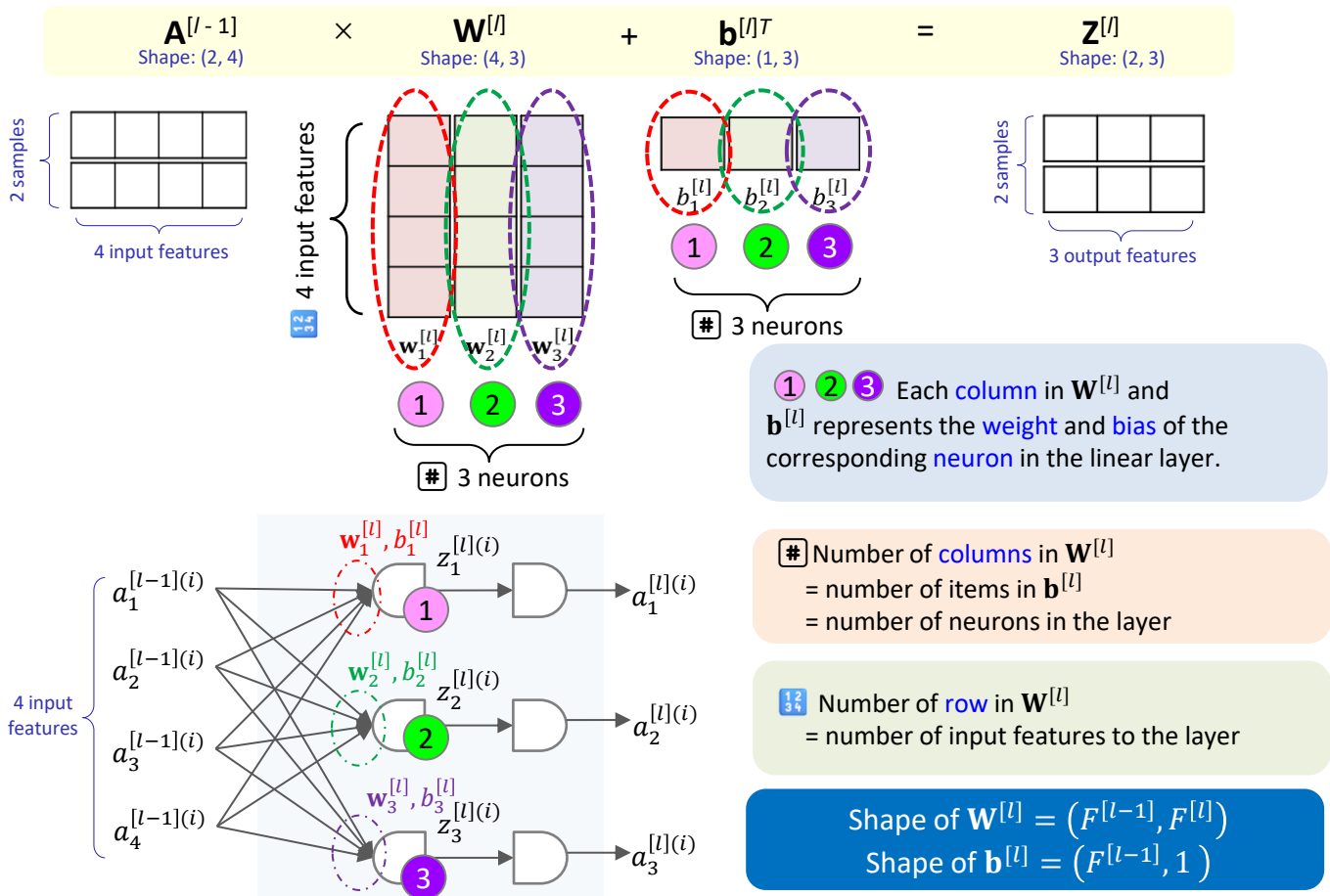
$$\mathbf{z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]T}$$



Summation Phase (Understanding the weight and bias)

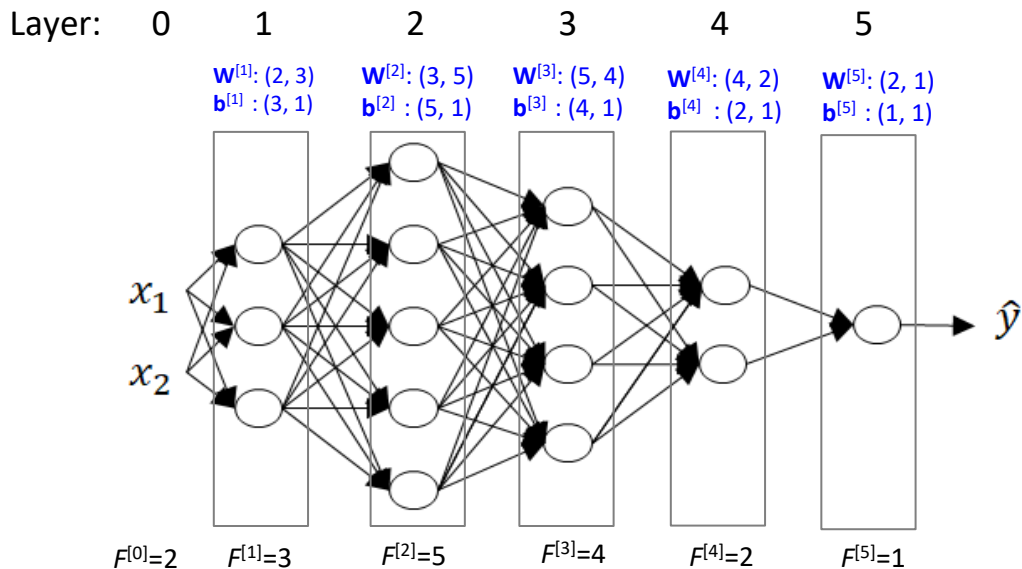


Summation Phase (Understanding the weight and bias)



Example:

Shape of $\mathbf{W}^{[l]} = (F^{[l-1]}, F^{[l]})$
Shape of $\mathbf{b}^{[l]} = (F^{[l-1]}, 1)$



PyTorch Linear Layer



Linear layer

`torch.nn.Linear`(in_features, out_features)

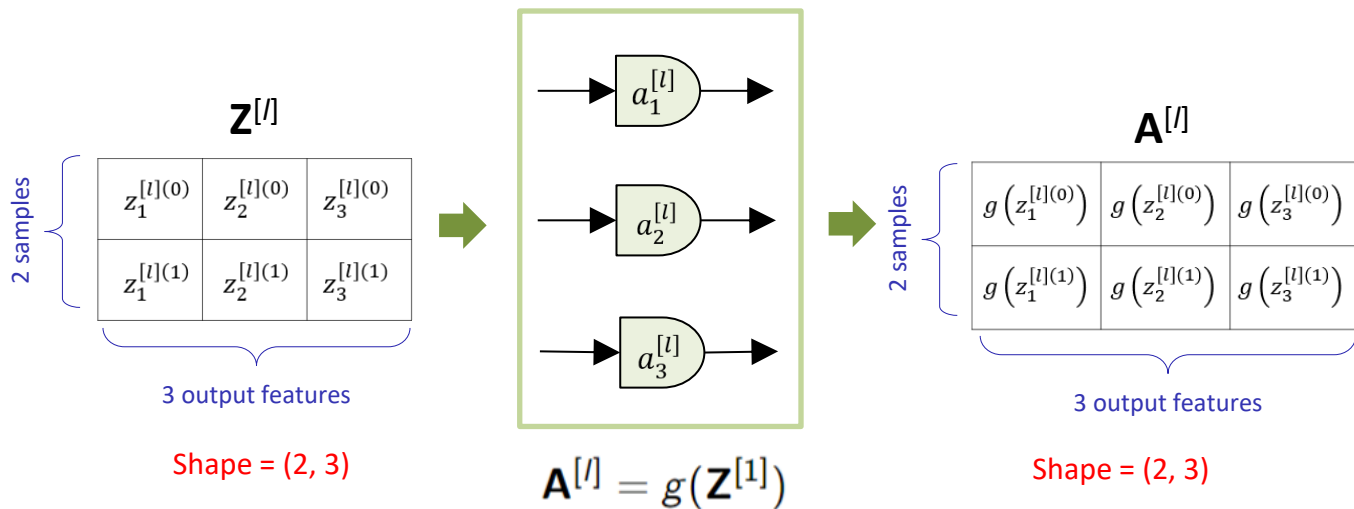
- Applies linear transformation to the incoming data
- Input: Tensor of shape (B, F_{in})
- Output: Tensor of shape (B, F_{out})
- Arguments:
 - `in_features`: size of each input sample
 - `out_features`: size of each output sample

Implementing a linear layer

```
>>> fc = nn.Linear(256, 10)
>>> x = torch.randn(batch_size, 256)
>>> y = fc(x)
>>> print(y)
```

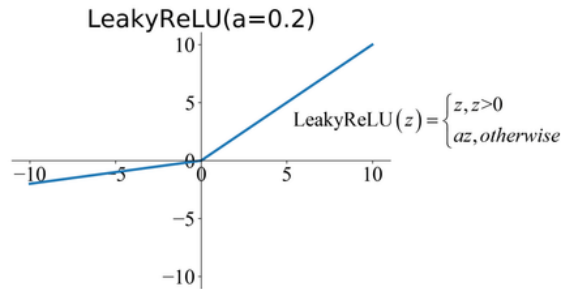
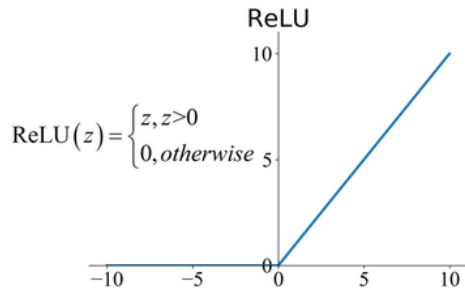
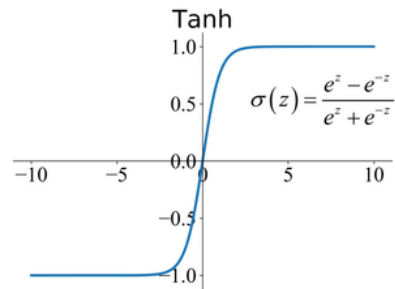
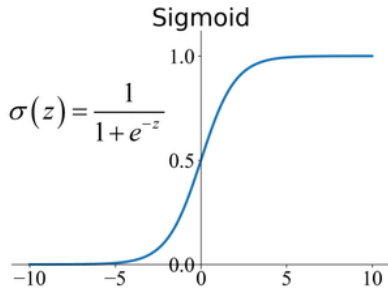
Activation Functions

- The activation function $g(\mathbf{Z}^{[l]})$ transforms the weighted input $\mathbf{Z}^{[l]}$ **non-linearly** to generate the activation output $\mathbf{A}^{[l]}$. It is typically an **element-wise** operation.
- $\mathbf{A}^{[l]}$ has the **same shape** as $\mathbf{Z}^{[l]}$.



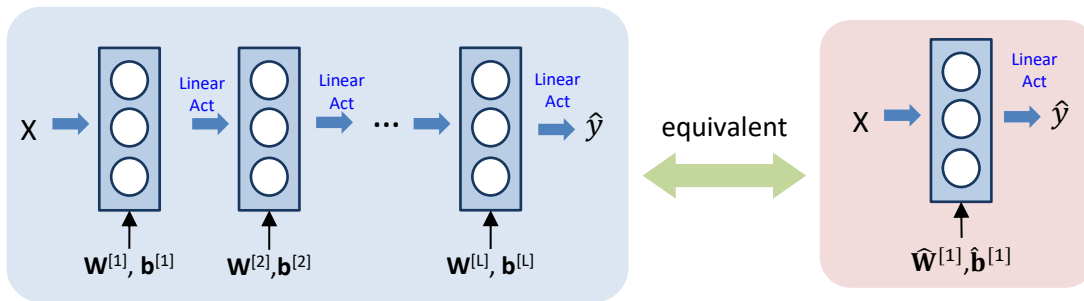
Activation Phase

- Different activation functions can be used



Why non-linear activation?

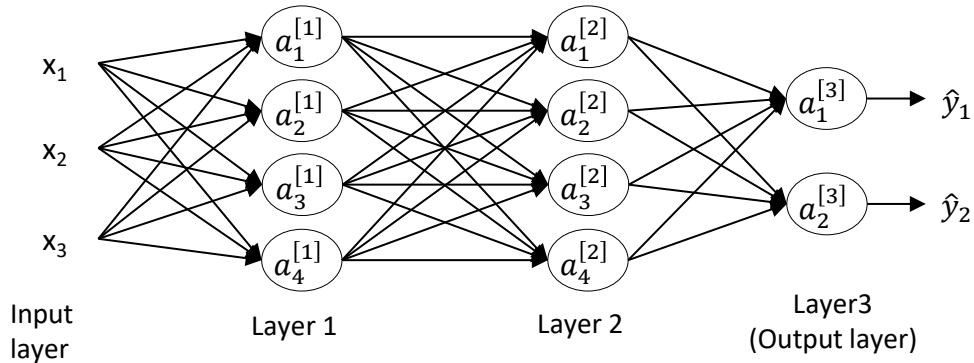
- A very deep neural network with linear activation collapses to a linear function.



$$\begin{aligned}\hat{\mathbf{Y}} &= \mathbf{M}^{[2]}(\mathbf{W}^{[2]}\mathbf{A}^{[1]} + \mathbf{b}^{[2]}) \\ &= \mathbf{M}^{[2]}\mathbf{W}^{[2]}\mathbf{A}^{[1]} + \mathbf{M}^{[2]}\mathbf{b}^{[2]} \\ &= \mathbf{M}^{[2]}\mathbf{W}^{[2]}\mathbf{M}^{[1]}(\mathbf{W}^{[1]}\mathbf{X} + \mathbf{b}^{[1]}) + \mathbf{M}^{[2]}\mathbf{b}^{[2]} \\ &= \boxed{\mathbf{M}^{[2]}\mathbf{W}^{[2]}\mathbf{M}^{[1]}\mathbf{W}^{[1]}} \mathbf{X} + \boxed{\mathbf{M}^{[2]}\mathbf{W}^{[2]}\mathbf{M}^{[1]}\mathbf{b}^{[1]} + \mathbf{M}^{[2]}\mathbf{b}^{[2]}} \\ &= \hat{\mathbf{W}}\mathbf{X} + \hat{\mathbf{b}}\end{aligned}$$

Activation outputs at each layer

- Each sample generates an **activation vector** in each layer
- Length of activation vector = #nodes in the layer



$$\mathbf{x}^{(1)} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \end{bmatrix}$$

$$\mathbf{x}^{(2)} = \begin{bmatrix} x_1^{(2)} & x_2^{(2)} & x_3^{(2)} \end{bmatrix}$$

$$\mathbf{X} \text{ or } \mathbf{A}^{[0]}$$

$$(B, F_x)$$

$$\mathbf{a}^{1} = \begin{bmatrix} a_1^{1} & a_2^{1} & a_3^{1} & a_4^{1} \end{bmatrix}$$

$$\mathbf{a}^{[1](2)} = \begin{bmatrix} a_1^{[1](2)} & a_2^{[1](2)} & a_3^{[1](2)} & a_4^{[1](2)} \end{bmatrix}$$

$$\mathbf{A}^{[1]}$$

$$(B, F^{[1]})$$

$$\mathbf{a}^{[2](1)} = \begin{bmatrix} a_1^{[2](1)} & a_2^{[2](1)} & a_3^{[2](1)} & a_4^{[2](1)} \end{bmatrix}$$

$$\mathbf{a}^{2} = \begin{bmatrix} a_1^{2} & a_2^{2} & a_3^{2} & a_4^{2} \end{bmatrix}$$

$$\mathbf{A}^{[2]}$$

$$(B, F^{[2]})$$

$$\hat{\mathbf{y}}^{(1)} = \begin{bmatrix} \hat{y}_1^{(1)} & \hat{y}_2^{(1)} \end{bmatrix}$$

$$\hat{\mathbf{y}}^{(2)} = \begin{bmatrix} \hat{y}_1^{(2)} & \hat{y}_2^{(2)} \end{bmatrix}$$

$$\hat{\mathbf{Y}} \text{ or } \mathbf{A}^{[3]}$$

$$(B, F_y)$$

Activation Functions in PyTorch

- Commonly used activation functions:

Activation Functions

```
>>> torch.sigmoid(x)
```

```
>>> torch.tanh(x)
```

```
>>> torch.relu(x)
```

```
>>> torch.leaky_relu(x)
```

Example

Given the parameters of a fully connected (FC) layer, $\mathbf{W}$ and $\mathbf{b}$:

$$\mathbf{W} = \begin{bmatrix} 0.2 & -0.5 & 0.1 & 2 \\ 1.5 & 1.3 & 2.1 & 0 \\ 0 & 0.25 & 0.2 & -0.3 \end{bmatrix} \quad \mathbf{b}^T = [1.1 \quad 3.2 \quad -1.2 \quad -2]$$

- a. Consider a single sample. What is the dimensionality of the input vector $\mathbf{a}^{[l-1]}$ and output vector $\mathbf{a}^{[l]}$?

- b. Given the following input matrix:

$$\mathbf{A}^{[l-1]} = \begin{bmatrix} 200 & 25 & 105 \\ 120 & 250 & 32 \end{bmatrix}$$

- How many samples are there? How many features do each sample have?
- Transform $\mathbf{A}^{[l-1]}$ with the FC layer above.

Types of Classification

Types of Classifications

- Binary Classification

cat:



1



0



0

- Multi-label Classification

cat, dog, monkey, man:



1,0,0,1



0,1,1,0



1,1,0,0

- Multi-class Classification

cat, dog, monkey, man:



1,0,0,0



0,0,1,0



0,1,0,0

Output layer and cost function

- The **output layer** and **cost function** needs to be set based on the type of task

Classification Task	#neurons in output layer	Activation	Cost function
Binary Classification	1	Sigmoid	Binary Cross Entropy (BCE)
Multi-label Classification	#class	Sigmoid	Average Binary Cross Entropy
Multi-class Classification	#class	softmax	Negative Log Likelihood (NLL)
		-	Cross Entropy (softmax + NLL)

Binary Classification (1/2)

- Ground truth is **binary** value (e.g., 1: cat, 0: non-cat)



$y = 1$



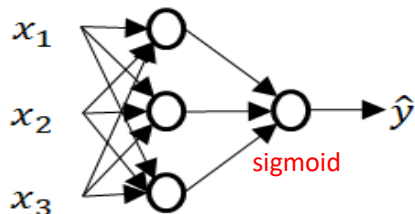
$y = 0$



$y = 0$

- Set up:

Output layer	#neurons = 1, sigmoid activation
Cost function	Binary cross entropy (BCE)



$$p(\text{cat}=1 | x)$$

$$p(\text{cat}=1 | x) + p(\text{cat}=0 | x) = 1$$

The output can be interpreted as the probability of the sample being positive.

Binary Classification (2/2)

- For binary classification, use the **binary cross entropy** (BCE) cost function.

$$\begin{aligned} J(\mathbf{w}, b) &= \frac{1}{B} \sum_{i=1}^B -y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) \\ &= \text{mean}(-\mathbf{y} * \ln \hat{\mathbf{y}} - (1 - \mathbf{y}) * \ln(1 - \hat{\mathbf{y}})) \end{aligned}$$

Multi-Label Classification (1/2)

- Number of class $C > 2$, **multiple classes** may exist in a sample.
Targeted class label is a binary vector.

cat
dog
bear
person



$\mathbf{y} = [0, 1, 0, 1]$



$\mathbf{y} = [1, 0, 0, 1]$



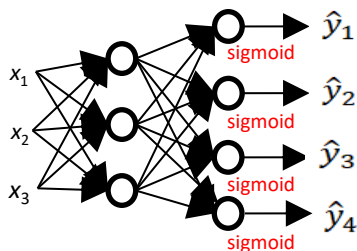
$\mathbf{y} = [1, 1, 0, 1]$



$\mathbf{y} = [0, 0, 0, 0]$

- Set up:

Output layer	#neurons = #classes, sigmoid activation.
Cost Function	Average binary cross entropy for all classes



$p(\text{cat}=1 x)$	$p(\text{cat} x) + p(\text{not cat} x) = 1$
$p(\text{dog}=1 x)$	$p(\text{dog} x) + p(\text{not dog} x) = 1$
$p(\text{bear}=1 x)$	$p(\text{bear} x) + p(\text{not bear} x) = 1$
$p(\text{person}=1 x)$	$p(\text{person} x) + p(\text{not person} x) = 1$

Each neuron is a binary classifier for the corresponding class.

Multi-Label Classification (2/2)

For multi-label classification, use the average of binary cross entropy of all classes

$$\begin{aligned} J(\mathbf{W}, \mathbf{b}) &= \frac{1}{C \times B} \sum_{c=1}^C \sum_{i=1}^B -y_c^{(i)} \ln \hat{y}_c^{(i)} - (1 - y_c^{(i)}) \ln (1 - \hat{y}_c^{(i)}) \\ &= \text{mean}(-\mathbf{Y} * \ln \hat{\mathbf{Y}} - (1 - \mathbf{Y}) * \ln(1 - \hat{\mathbf{Y}})) \end{aligned}$$

Multi-Class Classification (1/9)

- Number of class $C > 2$, **one class** per sample. Targeted class label is a one hot vector.

chick
dog
cat
others



$\mathbf{y} = [1, 0, 0, 0]$



$\mathbf{y} = [0, 0, 1, 0]$



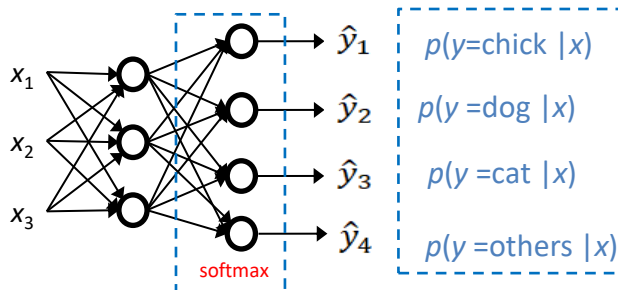
$\mathbf{y} = [0, 1, 0, 0]$



$\mathbf{y} = [0, 0, 0, 1]$

- Set up:

Output layer	#neurons = #classes Softmax activation	#neurons = #classes No activation	#neurons = #classes LogSoftMax activation
Cost Function	Negative Log Likelihood (NLL)	Cross Entropy (CE) (Softmax $\rightarrow$ NLL)	NLLLoss (Negate $\rightarrow$ Average)



The whole output layer functions as a single multi-class classifier. The output is a probability distribution:

$$p(y = \text{other} | x) + p(y = \text{cat} | x) + p(y = \text{dog} | x) + p(y = \text{chick} | x) = 1$$

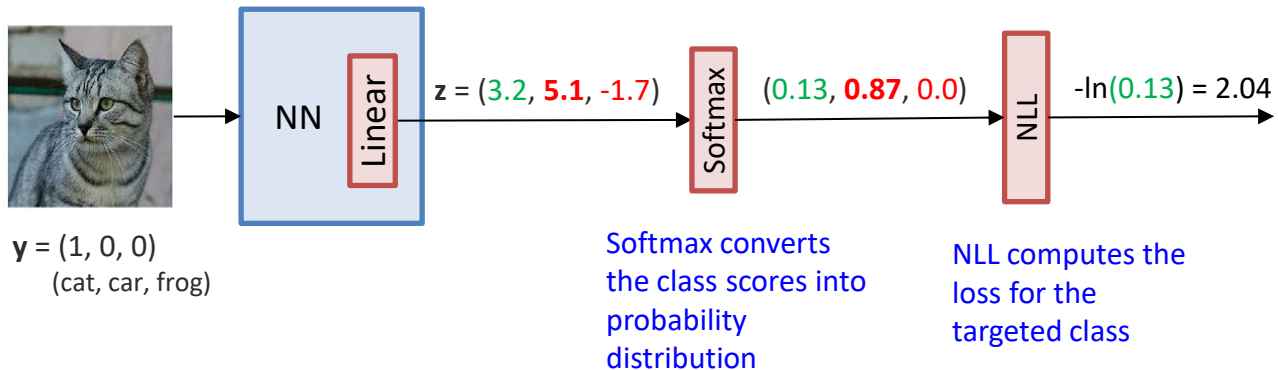
Multi-Class Classification (2/9)

Cross Entropy (CE) Cost

- Use **cross entropy** (CE) or **softmax cost** for multi-class classification.

Cross entropy Loss = softmax $\rightarrow$ negative log likelihood (NLL)

Example for single sample:

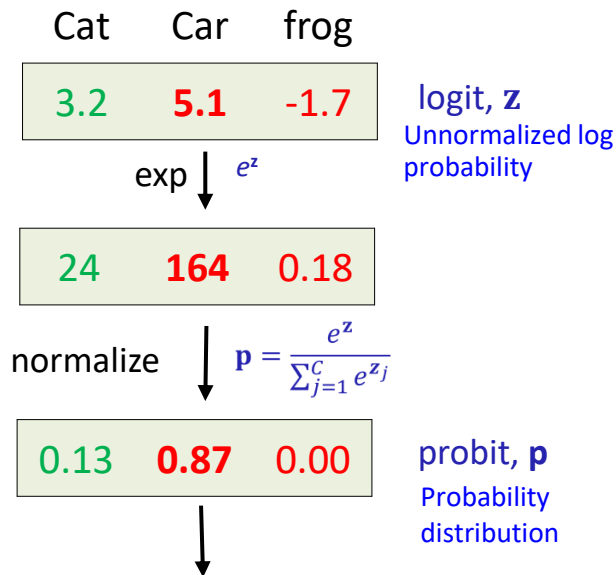


Multi-Class Classification (3/9)

Softmax function

- Converts an input vector into a probability distribution.
- Input is the $\mathbf{z}$ of the last layer which known as the **logit** or class score)
- The output of softmax $\mathbf{p}$ is also called a **probit** (a probability distribution).
- The network output $\hat{\mathbf{y}}$ can be either the logit or probit.

$$\text{Probit for class } i: p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$



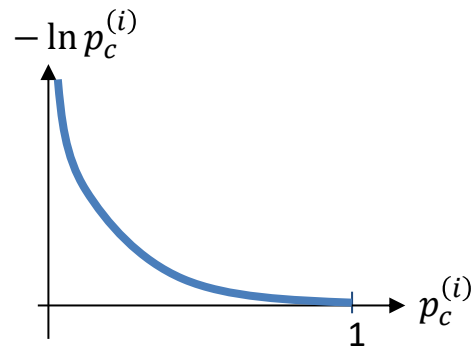
Multi-Class Classification (4/9)

Negative Log Likelihood

- The cost is the average negative (natural) log of the probit of the correct class.

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -\ln p_c^{(i)}$$

Notes: c is the index of the groundtruth label



Example:

Given the following sample:



	Cat	Car	frog
$\mathbf{y} =$	1	0	0

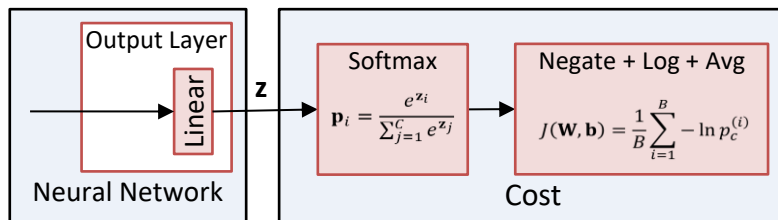
What is the cross entropy loss if the NN outputs the probit value $\mathbf{p} = [0.13, 0.87, 0.00]$?

$$\text{NLL} = -\ln(0.13) = 2.04$$

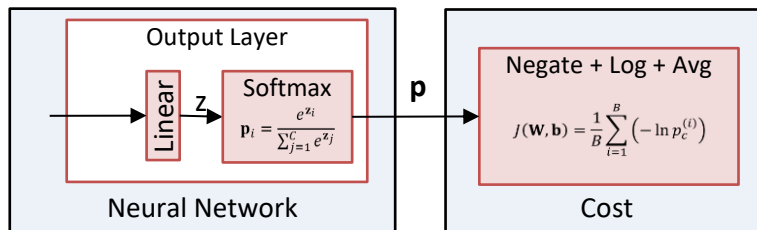
Multi-Class Classification (6/9)

Cross Entropy Cost Implementation Methods:

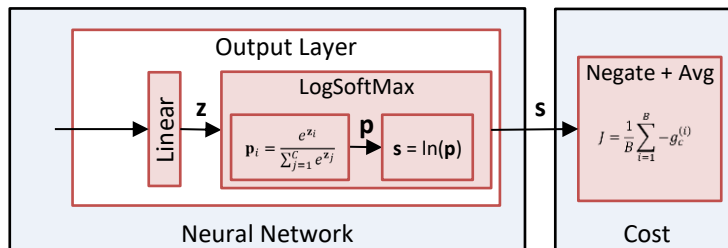
1. No activation (output layer) →
Cross entropy loss



2. Softmax activation (output layer) →
Negate + Log
Likelihood



3. LogSoftmax activation (output layer) → Negate
(Implemented by *NLLLoss* in PyTorch)



Multi-Class Classification (8/9)

Example 1: Given the logit output and the label for some input to the neural network:

$$\mathbf{Y} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad \text{logit} = \begin{bmatrix} 11 & 10 & -2 \\ 4 & 5 & 6.5 \\ 2.3 & -0.2 & 2.1 \\ 1.9 & 1.2 & -0.3 \end{bmatrix}$$

Solution:

$$\text{Probit} = \begin{bmatrix} \text{softmax}([11 & 10 & -2]) \\ \text{softmax}([4 & 5 & 6.5]) \\ \text{softmax}([2.3 & -0.2 & 2.1]) \\ \text{softmax}([1.9 & 1.2 & -0.3]) \end{bmatrix} = \begin{bmatrix} 0.73 & \mathbf{0.27} & 0.00 \\ 0.06 & 0.17 & \mathbf{0.77} \\ 0.53 & 0.04 & \mathbf{0.43} \\ \mathbf{0.62} & 0.31 & 0.07 \end{bmatrix}$$

$$\begin{aligned} \text{Cross entropy cost} &= (-\ln(0.27) - \ln(0.77) - \ln(0.43) - \ln(0.62))/4 \\ &= 0.72 \end{aligned}$$

Multi-Class Classification (9/9)

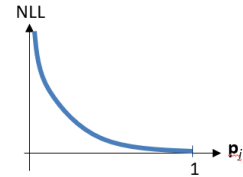
- What is the min/max possible loss for cross entropy loss?

$$\text{min loss} = 0$$

$$\text{max loss} = \text{inf}$$

$$\text{when } p(\text{correct class}) = 1$$

$$\text{when } p(\text{correct class}) = 0$$



- If the model parameters $\mathbf{W}$, $\mathbf{b}$ have very small values, what is the expected CE cost? Hints: when $\mathbf{W}$ is small, all items in $\mathbf{z}$ is small and almost uniform.

The probability of all classes are almost equal

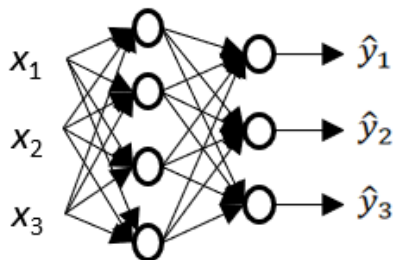
$$p_c = 1/C$$

$$J = -\ln(1/C)$$

Shallow Neural Network

Inference (vectorized)

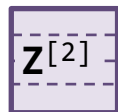
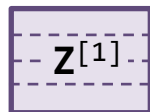
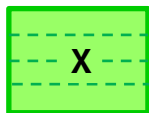
- Consider $L = 2$, B samples, vectorized version:



$B \times 4$

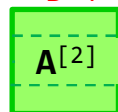
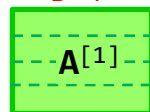
$B \times 4$

$B \times 3$



$B \times 4$

$B \times 4$



(a batch of
samples in
each iteration)

Forward Propagation ($L=2$, vectorized)

Forward prop through layer 1

$$\mathbf{Z}^{[1]} = \mathbf{X}\mathbf{W}^{[1]} + \mathbf{b}^{[1]T}$$

$B \times 4$ $B \times 3$ 3×4 1×4

$$\mathbf{A}^{[1]} = g(\mathbf{Z}^{[1]})$$

$B \times 4$ $B \times 4$

Forward prop through layer 2

$$\mathbf{Z}^{[2]} = \mathbf{A}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]T}$$

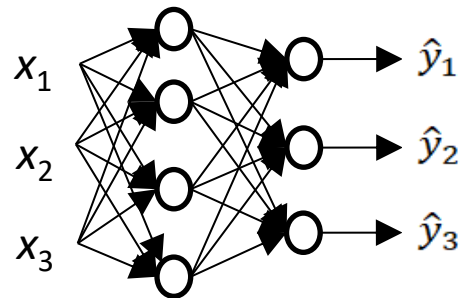
$B \times 3$ $B \times 4$ 4×3 1×3

$$\mathbf{A}^{[2]} = g(\mathbf{Z}^{[2]})$$

$B \times 3$ $B \times 3$

Gradient Descent

- Parameters: $\mathbf{W}^{[1]}$, $\mathbf{b}^{[1]}$, $\mathbf{W}^{[2]}$, $\mathbf{b}^{[2]}$



Gradient Descent

1. Initialize $\mathbf{W}^{[1]}$, $\mathbf{b}^{[1]}$, $\mathbf{W}^{[2]}$, $\mathbf{b}^{[2]}$
2. Repeat until convergence {
3. Forward propagation to compute $\hat{y}_1, \hat{y}_2, \hat{y}_3$
4. Compute J
5. Backpropagation to compute $\frac{\partial J}{\partial \mathbf{W}^{[1]}}$, $\frac{\partial J}{\partial \mathbf{b}^{[1]}}$, $\frac{\partial J}{\partial \mathbf{W}^{[2]}}$ and $\frac{\partial J}{\partial \mathbf{b}^{[2]}}$
6. Update parameters:

$$\mathbf{W}^{[1]} = \mathbf{W}^{[1]} - \alpha \frac{\partial J}{\partial \mathbf{W}^{[1]}}$$

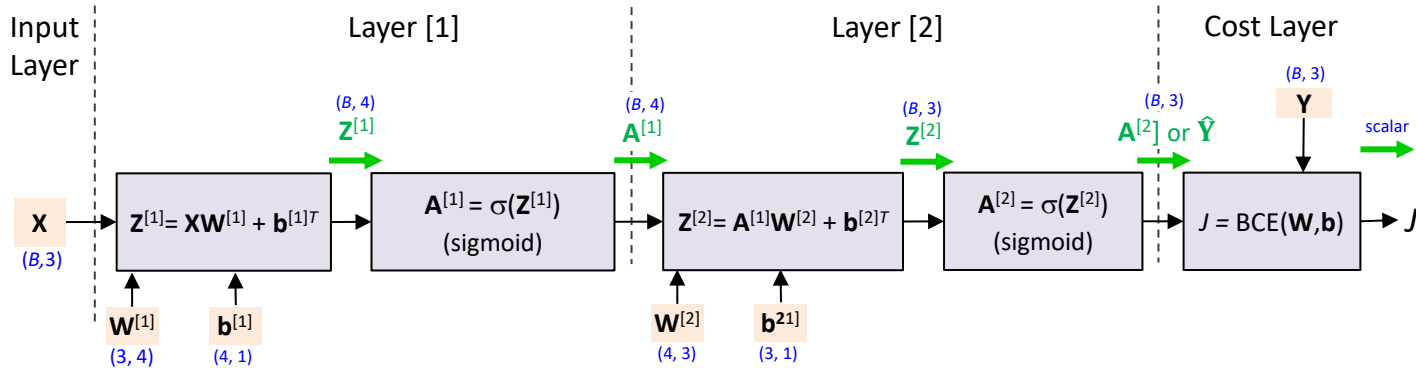
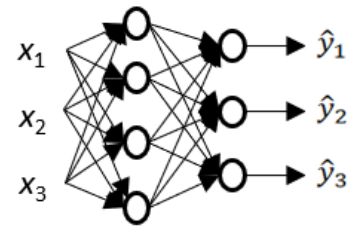
$$\mathbf{b}^{[1]} = \mathbf{b}^{[1]} - \alpha \frac{\partial J}{\partial \mathbf{b}^{[1]}}$$

$$\mathbf{W}^{[2]} = \mathbf{W}^{[2]} - \alpha \frac{\partial J}{\partial \mathbf{W}^{[2]}}$$

$$\mathbf{b}^{[2]} = \mathbf{b}^{[2]} - \alpha \frac{\partial J}{\partial \mathbf{b}^{[2]}}$$

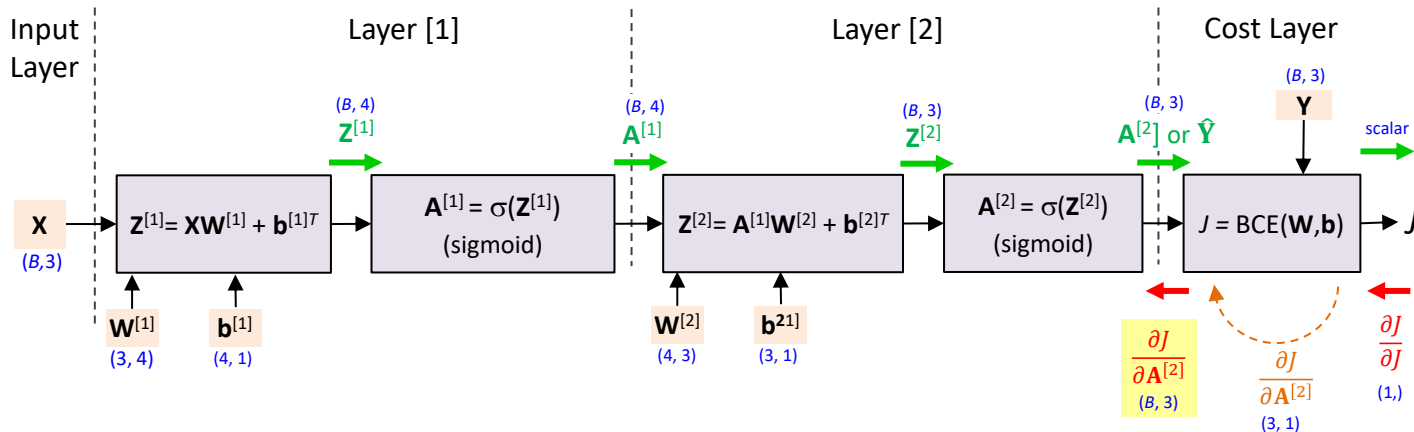
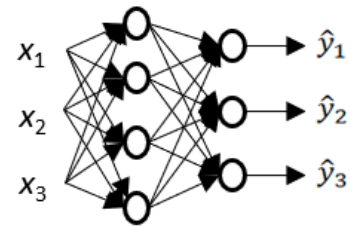
Example:

- Forward propagation



Example:

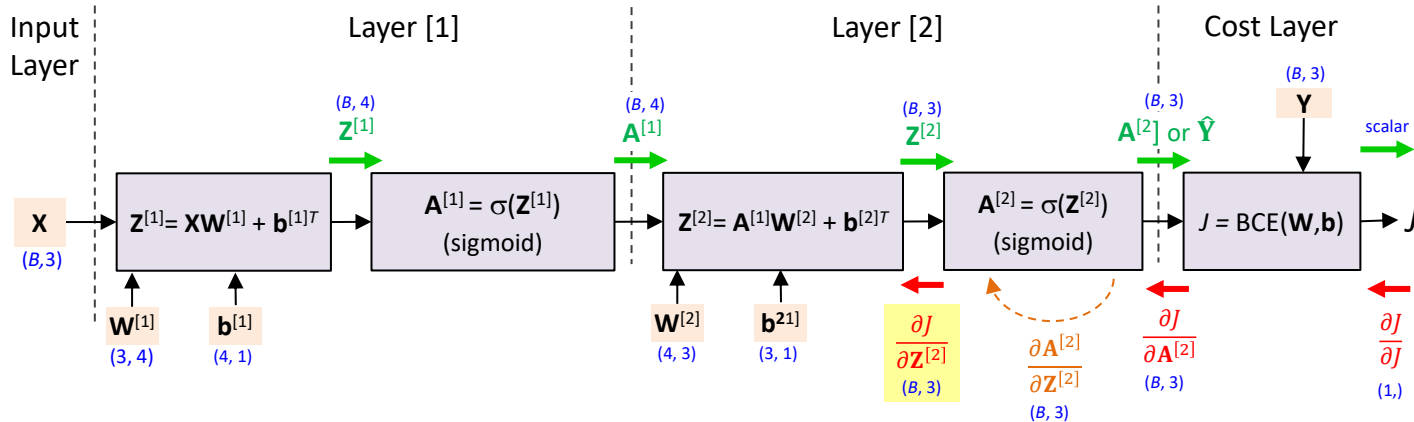
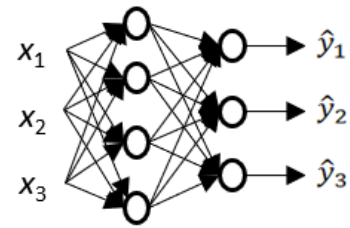
■ Backprop (Cost Layer)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial J} = 1$
Local Gradient	$\frac{\partial J}{\partial \mathbf{A}^{[2]}} = \frac{1}{C \times B} \left(\frac{1 - \mathbf{Y}}{1 - \mathbf{A}^{[2]}} - \frac{\mathbf{Y}}{\mathbf{A}^{[2]}} \right)$
Gradient at $\mathbf{A}^{[2]}$	$\frac{\partial J}{\partial \mathbf{A}^{[2]}} = \frac{1}{C \times B} \left(\frac{1 - \mathbf{Y}}{1 - \mathbf{A}^{[2]}} - \frac{\mathbf{Y}}{\mathbf{A}^{[2]}} \right)$

Example:

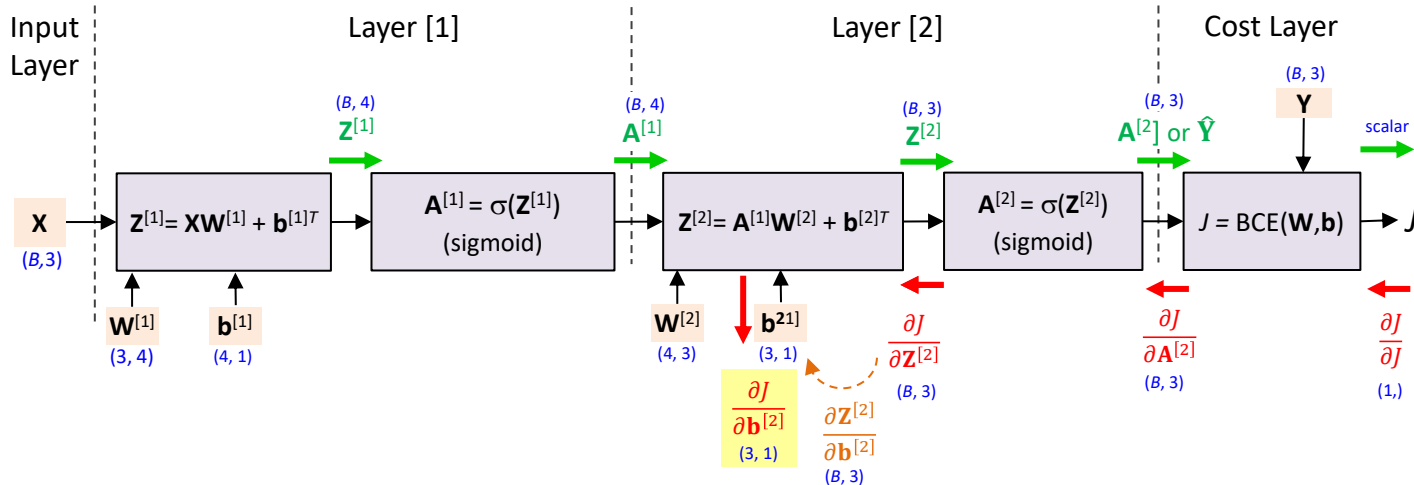
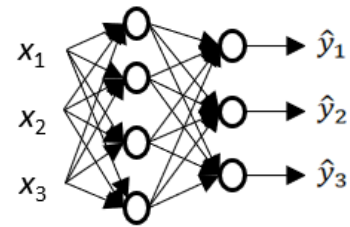
■ Backprop (Activation Layer 2)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{A}^{[2]}}$
Local Gradient	$\frac{\partial \mathbf{A}^{[2]}}{\partial \mathbf{z}^{[2]}} = \mathbf{A}^{[2]} * (1 - \mathbf{A}^{[2]})$
Gradient at $\mathbf{z}^{[2]}$	$\frac{\partial J}{\partial \mathbf{z}^{[2]}} = \frac{\partial J}{\partial \mathbf{A}^{[2]}} * \mathbf{A}^{[2]} * (1 - \mathbf{A}^{[2]})$

Example:

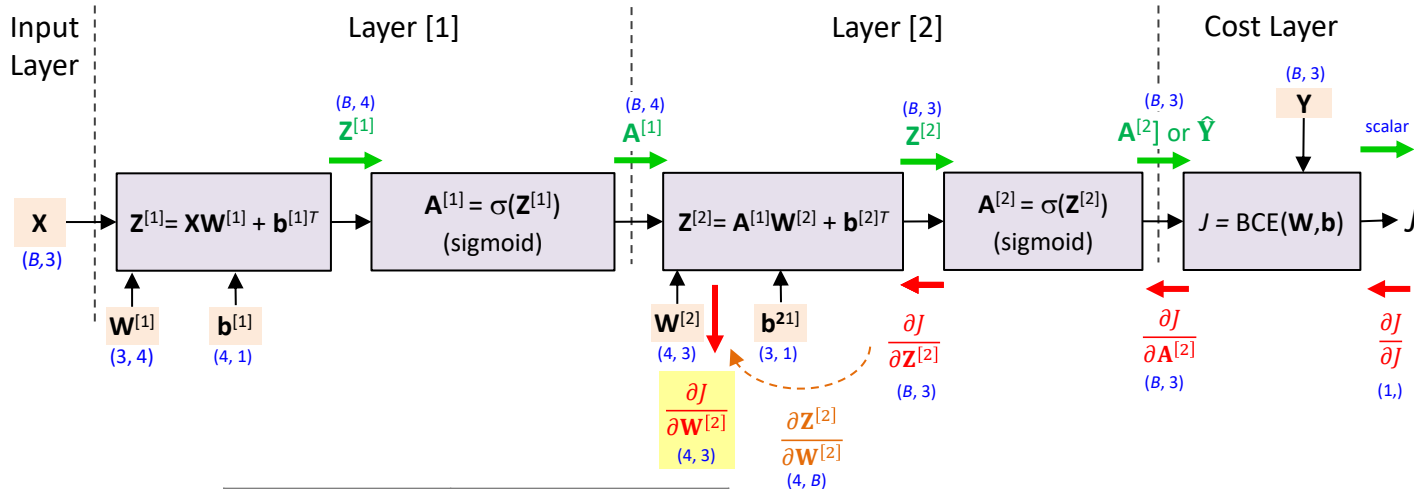
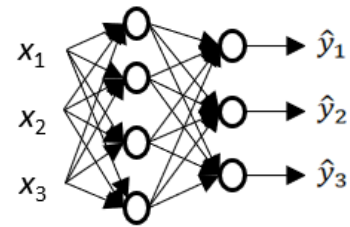
■ Backprop (Summation Layer 2)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{z}^{[2]}}$
Local Gradient	$\frac{\partial \mathbf{z}^{[2]}}{\partial \mathbf{b}^{[2]}} = \mathbf{1}_{B \times 3}$
Gradient at $\mathbf{b}^{[2]}$	$\frac{\partial J}{\partial \mathbf{b}^{[2]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}^{[2]}} \right)^T = \sum_{\text{axis}=0} \frac{\partial J}{\partial \mathbf{z}^{[2]}}$

Example:

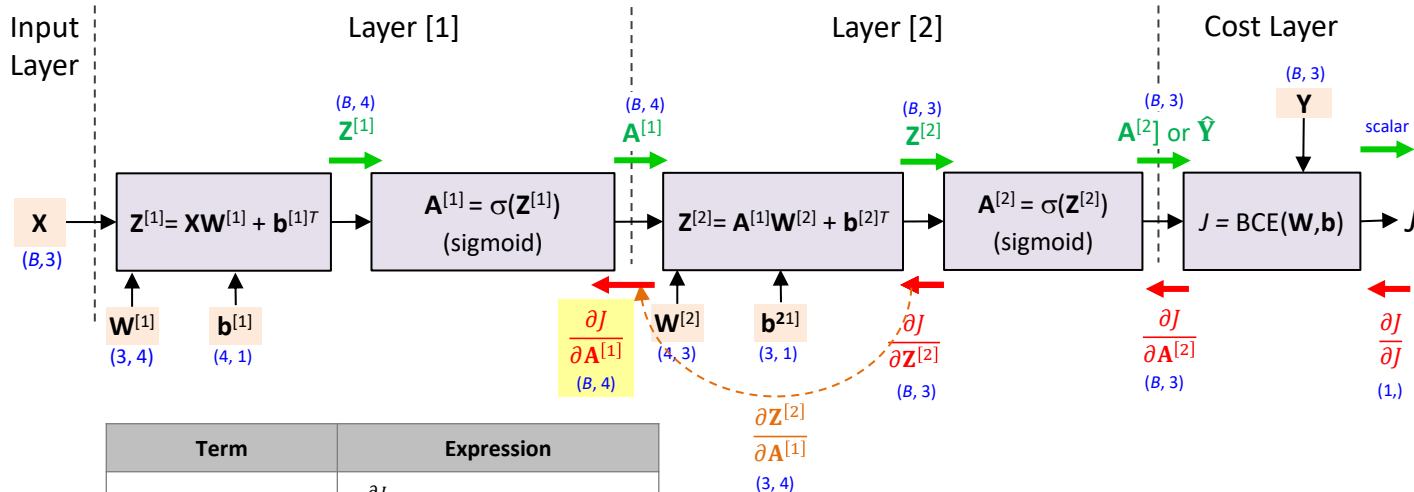
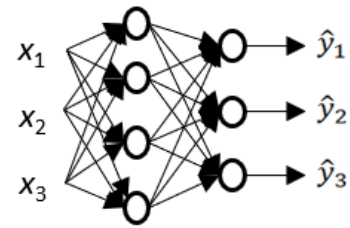
■ Backprop (Summation Layer 2)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{z}^{[2]}}$
Local Gradient	$\frac{\partial \mathbf{z}^{[2]}}{\partial \mathbf{W}^{[2]}} = \mathbf{A}^{[1]T}$
Gradient at $\mathbf{W}^{[2]}$	$\frac{\partial J}{\partial \mathbf{W}^{[2]}} = \mathbf{A}^{[1]T} \frac{\partial J}{\partial \mathbf{z}^{[2]}}$

Example:

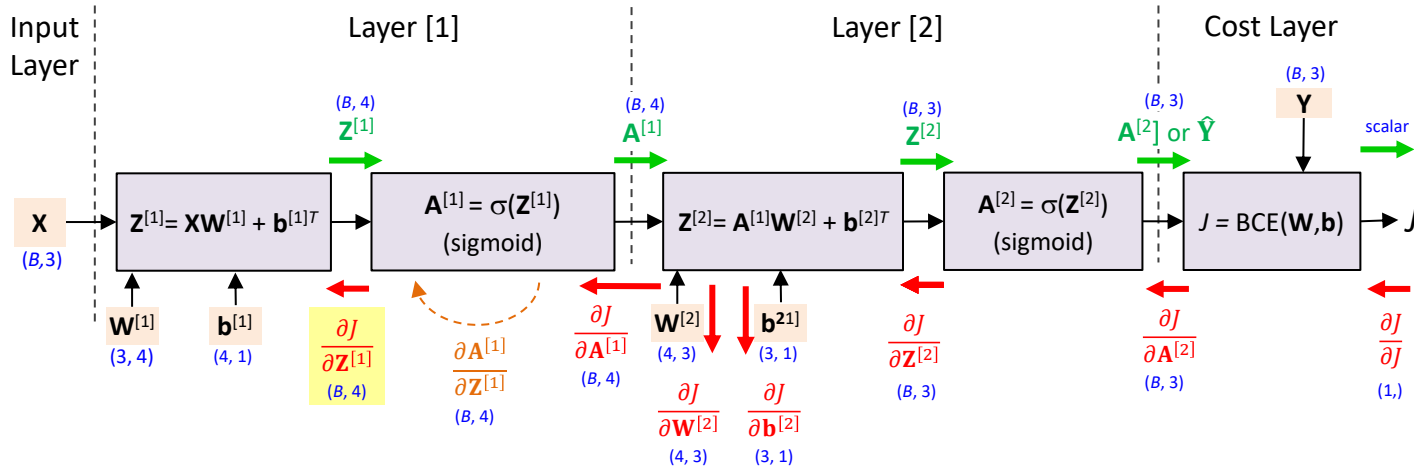
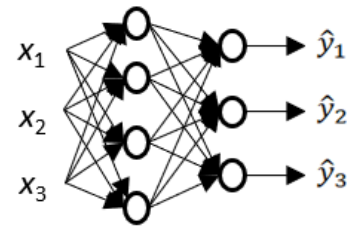
■ Backprop (Summation Layer 2)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{Z}^{[2]}}$
Local Gradient	$\frac{\partial \mathbf{Z}^{[2]}}{\partial \mathbf{A}^{[1]}} = \mathbf{W}^{[2]T}$
Gradient at $\mathbf{A}^{[1]}$	$\frac{\partial J}{\partial \mathbf{A}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \mathbf{W}^{[2]T}$

Example:

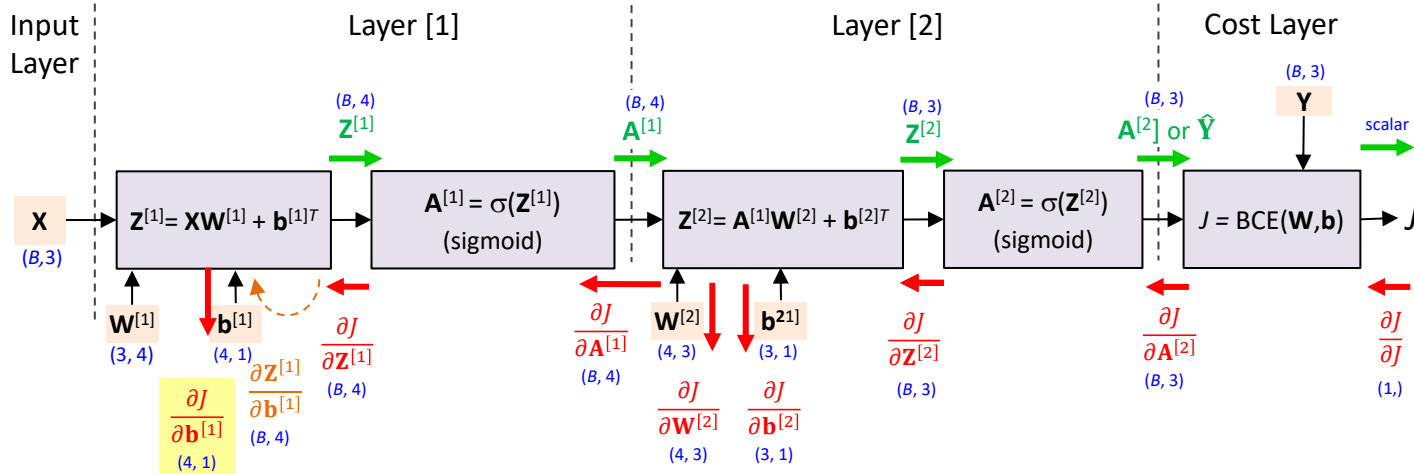
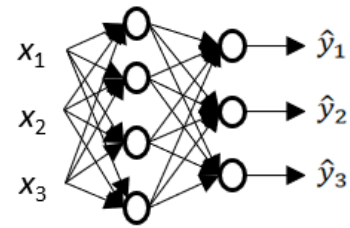
Backprop (Activation Layer 1)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{A}^{[1]}}$
Local Gradient	$\frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} = \mathbf{A}^{[1]} * (1 - \mathbf{A}^{[1]})$
Gradient at $\mathbf{Z}^{[1]}$	$\frac{\partial J}{\partial \mathbf{Z}^{[1]}} = \frac{\partial J}{\partial \mathbf{A}^{[1]}} * \mathbf{A}^{[1]} * (1 - \mathbf{A}^{[1]})$

Example:

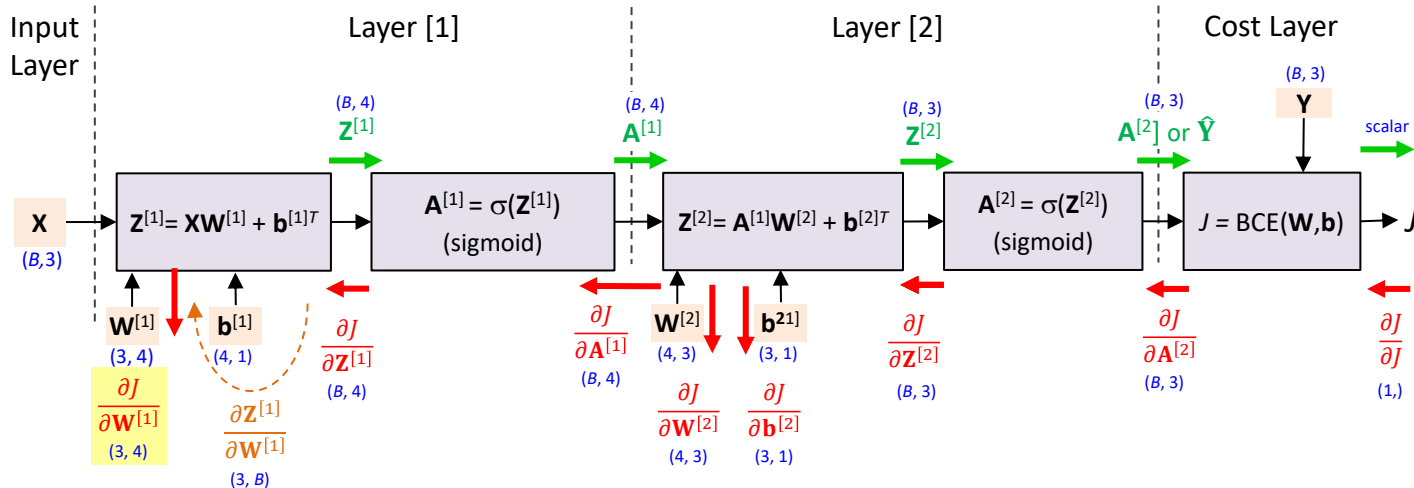
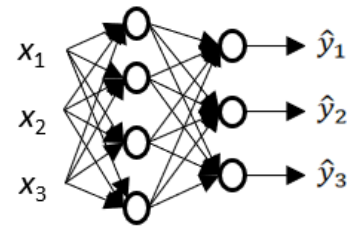
■ Backprop (Summation Layer 1)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{z}^{[1]}}$
Local Gradient	$\frac{\partial \mathbf{z}^{[1]}}{\partial \mathbf{b}^{[1]}} = \mathbf{1}_{B \times 4}$
Gradient at $\mathbf{b}^{[1]}$	$\frac{\partial J}{\partial \mathbf{b}^{[1]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}^{[1]}} \right)^T = \sum_{\text{axis}=0} \frac{\partial J}{\partial \mathbf{z}^{[1]}}$

Example:

■ Backprop (Summation Layer 1)



Term	Expression
Upstream gradient	$\frac{\partial J}{\partial \mathbf{Z}^{[1]}}$
Local Gradient	$\frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{W}^{[1]}} = \mathbf{X}^T$
Gradient at $\mathbf{W}^{[1]}$	$\frac{\partial J}{\partial \mathbf{W}^{[1]}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{Z}^{[1]}}$

Backpropagation

Backpropagation

Backpropagate through cost layer

1. Compute $\frac{\partial J}{\partial \bar{\mathbf{Y}}} = \frac{1}{C \times B} \left(\frac{1 - \mathbf{Y}}{1 - \bar{\mathbf{Y}}} - \frac{\mathbf{Y}}{\bar{\mathbf{Y}}} \right)$

Backpropagate through layer 2

2. Compute $\frac{\partial J}{\partial \mathbf{Z}^{[2]}} = \frac{\partial J}{\partial \bar{\mathbf{Y}}} * \mathbf{A}^{[2]} * (1 - \mathbf{A}^{[2]})$ # Activation phase

3. Compute $\frac{\partial J}{\partial \mathbf{W}^{[2]}} = \mathbf{A}^{[1]T} \frac{\partial J}{\partial \mathbf{Z}^{[2]}}$ # Summation phase

4. Compute $\frac{\partial J}{\partial \mathbf{b}^{[2]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \right)^T$

5. Compute $\frac{\partial J}{\partial \mathbf{A}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \mathbf{W}^{[2]T}$

Backpropagate through layer 1

7. Compute $\frac{\partial J}{\partial \mathbf{Z}^{[1]}} = \frac{\partial J}{\partial \mathbf{A}^{[1]}} * \mathbf{A}^{[1]} * (1 - \mathbf{A}^{[1]})$ # Activation phase

8. Compute $\frac{\partial J}{\partial \mathbf{W}^{[1]}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{Z}^{[1]}}$ # Summation phase

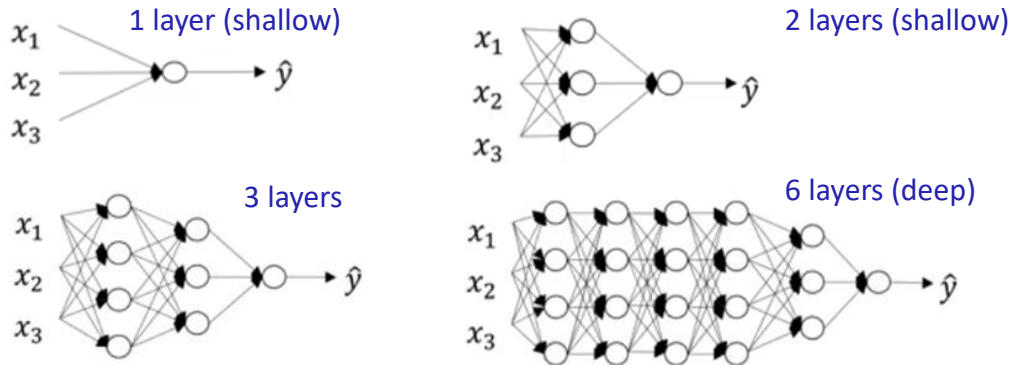
9. Compute $\frac{\partial J}{\partial \mathbf{b}^{[1]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[1]}} \right)^T$

10. Compute $\frac{\partial J}{\partial \mathbf{X}} = \frac{\partial J}{\partial \mathbf{Z}^{[1]}} \mathbf{W}^{[1]T}$

Deep Neural Network

What is a deep neural network?

- Degree of deepness refer to number of layers in the network



- **Higher capacity** – able to model more complex functions
- **More difficult to train** – takes more epochs and easier to overfit
- **Needs more training samples** – to reduce overfitting
- Difficult to know in advance **how deep**. Solution: cross-validation
- MLP is typically shallower than CNN

Building Blocks of MLP

- Implementation of each layer is similar → building blocks

Forward Propagation

Forward prop through layer 1

$$\mathbf{Z}^{[1]} = \mathbf{X}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$$

$$\mathbf{A}^{[1]} = g(\mathbf{Z}^{[1]})$$

forward_linear

Forward prop through layer 2

$$\mathbf{Z}^{[2]} = \mathbf{A}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}$$

$$\mathbf{A}^{[2]} = g(\mathbf{Z}^{[2]})$$

forward_linear

Backpropagation

Backpropagate through cost layer

$$1. \text{ Compute } \frac{\partial J}{\partial \mathbf{Y}} = \frac{1}{C \times B} \left(\frac{1 - \mathbf{Y}}{1 - \mathbf{Y}} - \frac{\mathbf{Y}}{\mathbf{Y}} \right)$$

Backpropagate through layer 2

$$2. \text{ Compute } \frac{\partial J}{\partial \mathbf{Z}^{[2]}} = \frac{\partial J}{\partial \mathbf{Y}} * \mathbf{A}^{[2]} * (1 - \mathbf{A}^{[2]})$$

$$3. \text{ Compute } \frac{\partial J}{\partial \mathbf{W}^{[2]}} = \mathbf{A}^{[1]T} \frac{\partial J}{\partial \mathbf{Z}^{[2]}}$$

$$4. \text{ Compute } \frac{\partial J}{\partial \mathbf{b}^{[2]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \right)^T$$

$$5. \text{ Compute } \frac{\partial J}{\partial \mathbf{A}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \mathbf{W}^{[2]T}$$

backward_linear

Backpropagate through layer 1

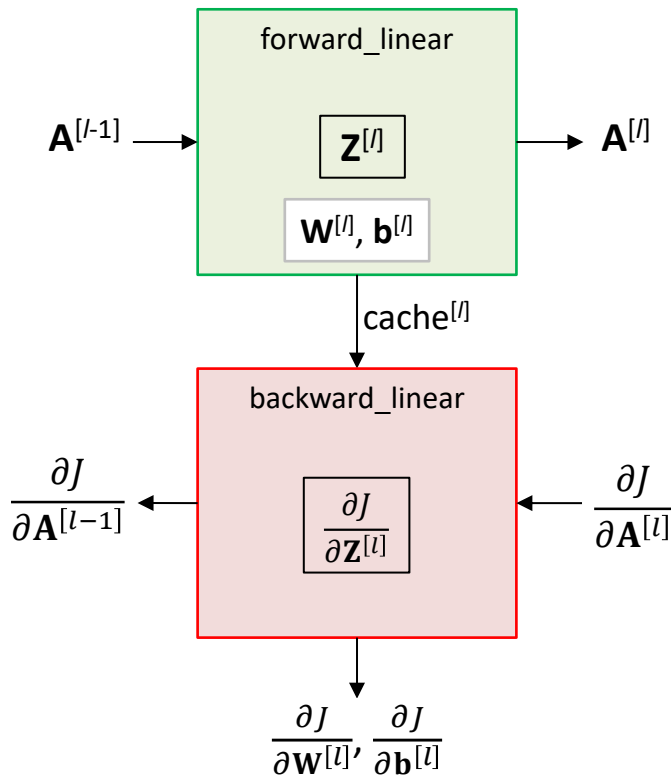
$$7. \text{ Compute } \frac{\partial J}{\partial \mathbf{Z}^{[1]}} = \frac{\partial J}{\partial \mathbf{A}^{[1]}} * \mathbf{A}^{[1]} * (1 - \mathbf{A}^{[1]})$$

$$8. \text{ Compute } \frac{\partial J}{\partial \mathbf{W}^{[1]}} = \mathbf{X}^T$$

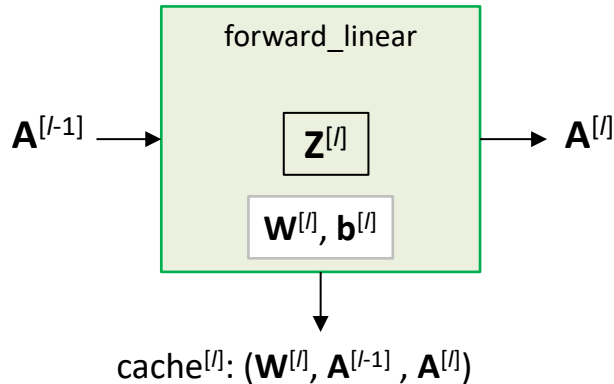
$$9. \text{ Compute } \frac{\partial J}{\partial \mathbf{b}^{[1]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[1]}} \right)^T$$

$$10. \text{ Compute } \frac{\partial J}{\partial \mathbf{X}} = \frac{\partial J}{\partial \mathbf{Z}^{[1]}} \mathbf{W}^{[1]T}$$

backward_linear



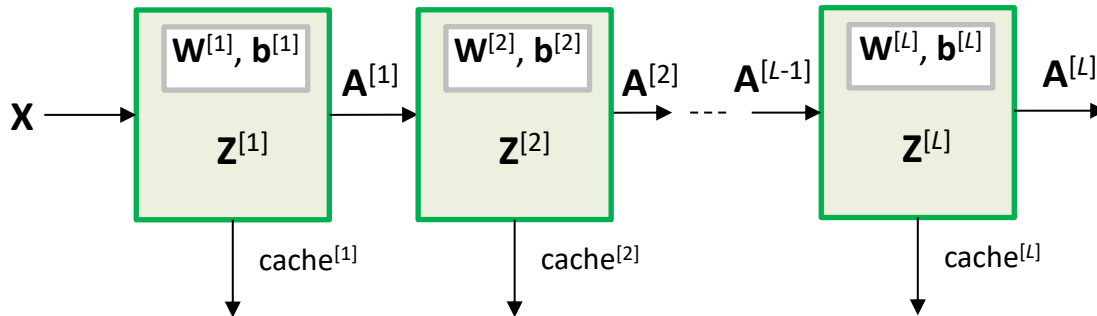
Forward propagation through 1 layer



forward_linear ($\mathbf{A}^{[l-1]}, \mathbf{W}^{[l]}, \mathbf{b}^{[l]}, \text{act_function}^{[l]}$)

1. Compute $\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}$
2. Compute $\mathbf{A}^{[l]} = \begin{cases} \frac{e^{\mathbf{Z}^{[l]}} - e^{-\mathbf{Z}^{[l]}}}{e^{\mathbf{Z}^{[l]}} + e^{-\mathbf{Z}^{[l]}}} & \text{if act_function}^{[l]} = \text{tanh} \\ \frac{1}{1 + e^{-\mathbf{Z}^{[l]}}} & \text{if act_function}^{[l]} = \text{sigmoid} \end{cases}$
3. Save $\mathbf{W}^{[l]}, \mathbf{A}^{[l-1]}, \mathbf{A}^{[l]}$ into cache^[l]
4. return $\mathbf{A}^{[l]}, \text{cache}^{[l]}$

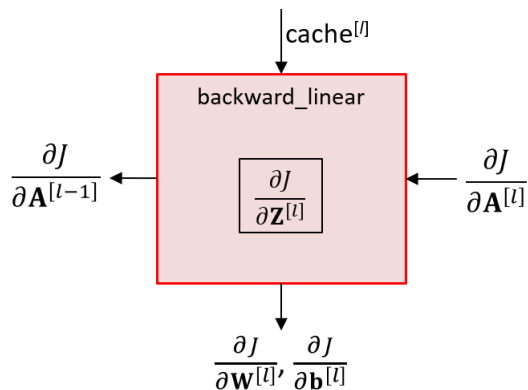
Forward propagation through L layers



`forward( $\mathbf{X}, \mathbf{W}, \mathbf{b}, \text{act\_function}$ )`

1. Set $\mathbf{A} = \mathbf{X}$ # setup the input
2. for $l = 1$ to L
3. $\mathbf{A}^{[l-1]} = \mathbf{A}^{[l]}$ # set the input to current layer
4. $\mathbf{A}^{[l]}, \text{cache}^{[l]} = \text{forward_linear}(\mathbf{A}^{[l-1]}, \mathbf{W}^{[l]}, \mathbf{b}^{[l]}, \text{act_function}^{[l]})$ # forward propagation
5. Save $\text{cache}^{[l]}$ into **caches** # save for backpropagation
6. Set $\hat{\mathbf{Y}} = \mathbf{A}^{[l]}$ # save predicted value
7. return $\hat{\mathbf{Y}}, \text{caches}$

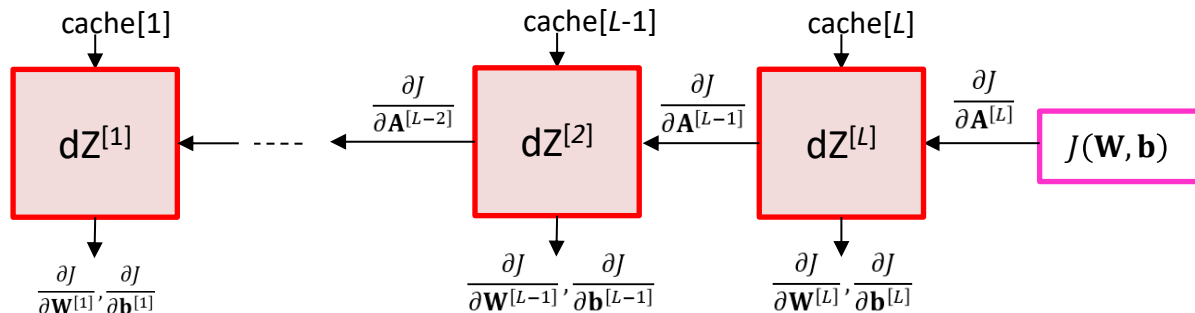
Backpropagation through 1 Layer



`backward_linear( $\frac{\partial J}{\partial \mathbf{A}^{[l]}}$ ,  $\text{cache}^{[l]}$ ,  $\text{activation}^{[l]}$ )`

1. Get $\mathbf{W}^{[l]}, \mathbf{A}^{[l-1]}, \mathbf{A}^{[l]}$ from cache
2. Compute $\frac{\partial J}{\partial \mathbf{Z}^{[l]}} = \begin{cases} \frac{\partial J}{\partial \mathbf{A}^{[l]}} * \mathbf{A}^{[l]} * (1 - \mathbf{A}^{[l]}) & \text{for sigmoid activation} \\ \frac{\partial J}{\partial \mathbf{A}^{[l]}} * (1 - \mathbf{A}^{[l]^2}) & \text{for tanh activation} \end{cases}$
3. Compute $\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \mathbf{A}^{[l-1]T} \frac{\partial J}{\partial \mathbf{Z}^{[l]}}$
4. Compute $\frac{\partial J}{\partial \mathbf{b}^{[l]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \right)^T$
5. Compute $\frac{\partial J}{\partial \mathbf{A}^{[l-1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \mathbf{W}^{[l]T}$
6. return $\frac{\partial J}{\partial \mathbf{A}^{[l-1]}}, \frac{\partial J}{\partial \mathbf{W}^{[l]}}, \frac{\partial J}{\partial \mathbf{b}^{[l]}}$

Backpropagation through L Layers



backward ($\mathbf{A}^{[L]}, \mathbf{Y}, \text{caches}$)

Backpropagate the cost layer

1. Compute $\frac{\partial J}{\partial \hat{\mathbf{Y}}} = \frac{1}{C \times B} \left(\frac{1 - \mathbf{Y}}{1 - \hat{\mathbf{Y}}} - \frac{\mathbf{Y}}{\hat{\mathbf{Y}}} \right)$

Compute gradient for layer L to 1

2. Set $\frac{\partial J}{\partial \mathbf{A}^{[L]}} = \frac{\partial J}{\partial \hat{\mathbf{Y}}}$ # Initialize the upstream gradient

3. for $l = L$ to 1:

4. Set $\frac{\partial J}{\partial \mathbf{A}^{[l]}} = \frac{\partial J}{\partial \mathbf{A}^{[l-1]}}$ # set up the upstream gradient for layer l

5. $\frac{\partial J}{\partial \mathbf{A}^{[l-1]}}, \frac{\partial J}{\partial \mathbf{W}^{[l]}}, \frac{\partial J}{\partial \mathbf{b}^{[l]}} = \text{backward_linear} \left(\frac{\partial J}{\partial \mathbf{A}^{[l]}}, \text{cache}^{[l]} \right)$ # backprop through layer l

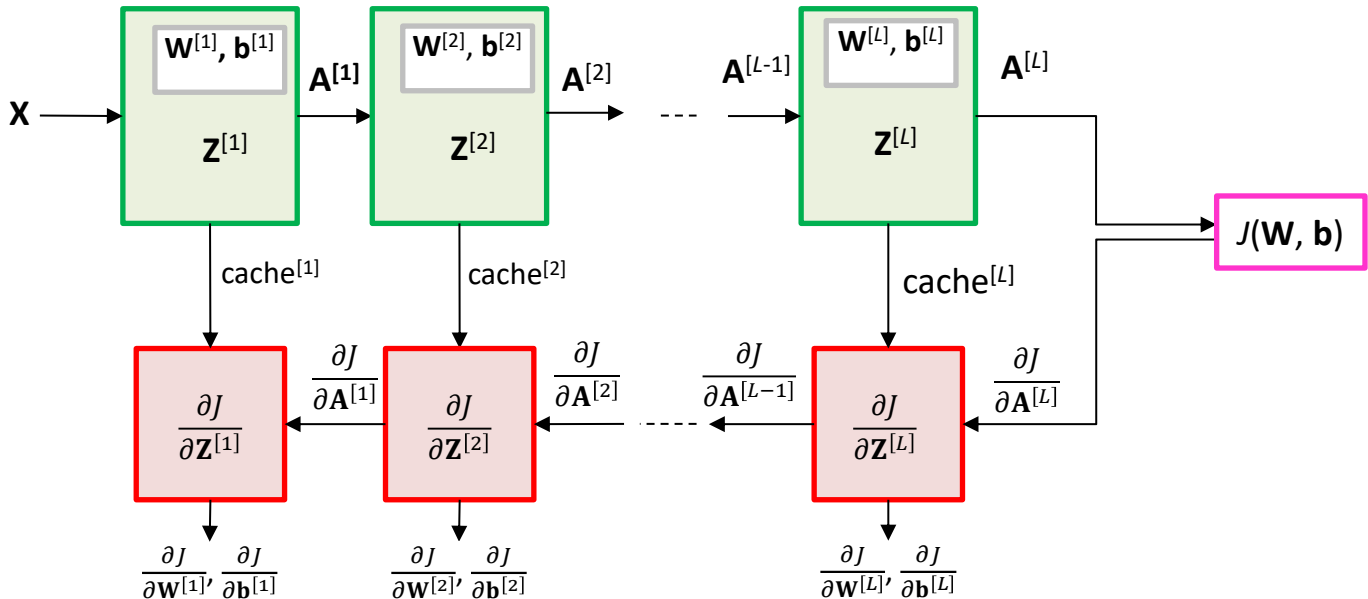
6. Save $\frac{\partial J}{\partial \mathbf{W}^{[l]}}$ into **weights**

7. Save $\frac{\partial J}{\partial \mathbf{b}^{[l]}}$ into **biases**

8. return **weights, biases**

Putting it together

- The complete implementation of the computational graph:



*Thank you
for your attention*